

MACOS LOCAL ARCHITECTURE

Agentic Engineering Framework

A step-by-step blueprint for durable multi-agent orchestration, repository contracts, and zero-cost local iteration.

The Single Core Vulnerability

0 KB

COMMITTED STATE ON DISK

Volatile Memory vs. Versioned State

When sessions live entirely inside chat windows and agent transcripts, every context compaction or token limit wipes the plan.

Agents do not fail from lack of reasoning capability; they fail from structural amnesia. By anchoring roadmaps, task scopes, and commands to disk, token exhaustion ceases to be a blocker.

The Four Fatal Failure Modes



Context Loss

Plans stored in chat windows vanish across sessions. Solved by version-controlled docs/`plan/` and mandatory handoff logs.



Unreviewable PRs

Agents invent runaway scope. Solved by a hard 400-line diff gate and upfront task briefs declared before writing any code.



Irreproducibility

Agents execute ad-hoc commands. Solved by forbidding arbitrary shell calls and routing all actions through committed `just` targets.

Five-Layer Governance Model

Structured Memory Over Agent Chat

This architecture decouples execution from conversation. Agents never coordinate directly with each other; they coordinate strictly through committed repository files.

You remain the sole authority (Layer 0), governing the repository contract (Layer 1), short-lived specialized workers (Layer 2), CI gates (Layer 3), and an audit ledger (Layer 4).



Local Mac Toolchain Setup

Command & Execution Core

- Install task runner and Git tools via `brew install just git gh`
- Initialize Rust toolchain for compiler speed and memory safety
- Enforce single-entry point: developer and agents run identical `justfile` targets

Multi-Provider Fallback Layer

- Install Ollama (`brew install ollama`) for 100% offline, zero-token runs
- Use Aider CLI as an alternative driver for local and open models
- Map role briefs to Claude Code slash commands in `.claude/commands/`

Layer 1: The Repository Contract



CLAUDE.md Universal Rule: Maintained under 150 lines. Every agent reads this file first. Forbids unapproved dependencies, main-branch pushes, and untested diffs.



The Mandatory Read-Back: The agent must state its phase, task brief, expected files, and diff estimate in under 10 lines—and wait for human approval before editing code.

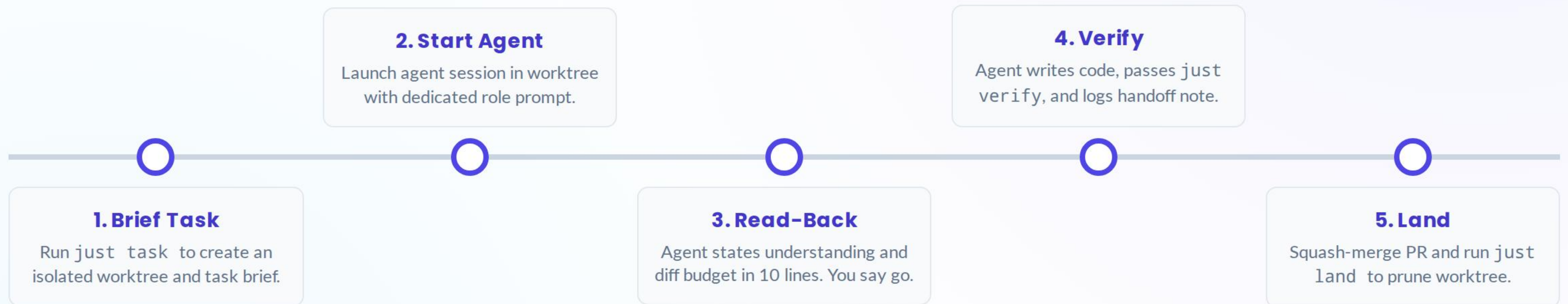


Structured Task Briefs: Every task begins from `.task/BRIEF.md` defining explicit in-scope and out-of-scope boundaries to stop scope explosion before it begins.

3-Slot Worktree Concurrency

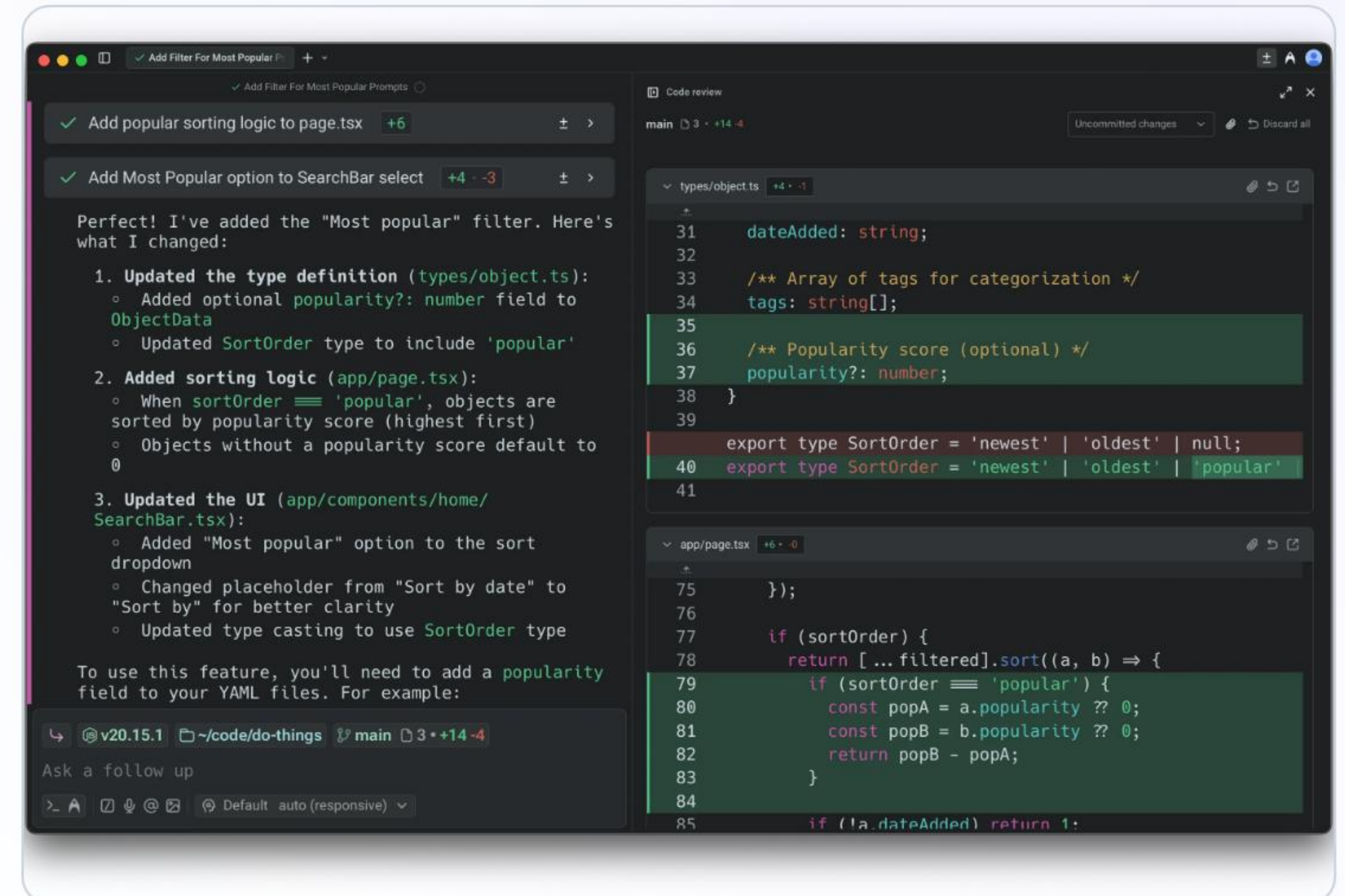
Slot	Domain	Target Boundary	Concurrency & Safety Rule
Slot 1	Deep Numerics	crates/ballistics	Strictly f64, CPU-only. Exactly 1 active task at a time; no parallel edits.
Slot 2	App Shell & UI	crates/app	Bevy/wgpu graphics. Decoupled from physics solver; parallel to Slot 1.
Slot 3	Infrastructure	.github/, justfile, docs/	CI gates, benchmark suites, and documentation. Runs parallel to Slots 1 & 2.

The Daily Five-Step Loop



Automated Definition of Done

- **Diff-Size Gate:** Hard failure if a pull request exceeds 400 lines (excluding fixtures and locks). Prevents giant unreviewable diffs.
- **Test-Integrity Gate:** Fails automatically if any test function is deleted, ignored, or if numerical tolerance is loosened.
- **Provenance Gate:** Blocks any physics changes lacking a corresponding handoff note and changelog entry.
- **Numerical Regression:** Enforces bit-accuracy against published reference firing fixtures.



Day One Rollout Schedule



Morning: Contract

Establish repo skeleton, copy CLAUDE.md, commit justfile, and configure main branch protections. (2 Hours)



Midday: Gates

Run CI Agent to stand up pr.yml and local verification bash gates. Validate loop on low-risk config. (2 Hours)



Afternoon: Physics

Architect decomposes Phase 0 into sub-400-line tasks. Test Agent creates published fixture; Ballistics Agent lands first solve. (3 Hours)

The Convergence Principle

“

Agentic development does not fail on capability. It fails on state. Every mechanism is a way of writing state to disk so amnesia stops mattering.

”

– Architecture Appendix A • Technical Report

Ready to Launch

Your local Mac environment is structured for continuous, autonomous progress.

- ✓ `justfile` worktree automation ready
- ✓ Layer 1 contract & task brief template configured
- ✓ CI regression & diff-budget gates defined
- ✓ Multi-provider local fallbacks (Ollama/Aider) available

Image Sources



https://www.logicmonitor.com/wp-content/uploads/2026/04/Blog_Deep-Dive_API-Architecture-Patterns-and-Best-Practices_940x600_Featured-Image-1.png

Source: www.logicmonitor.com



https://docs.warp.dev/_astro/code-review-panel-update.OokO3LEH_1Maom3.webp?dpl=dpl_2xadS6wYSsQFcwRfppt1erZKJtDi

Source: docs.warp.dev